Test report 7 – Kernel fixes and variable radius

SMLib facade, engine, language, CLI and browser UI · 2026-10-23 · Commit 19f2e47

cargo test --workspace	cargo clippy	Node (smlib.wasm)	Vitest	Playwright	OpenSCAD comparison
298/298 · 298/298 · 298/298 3 runs	**3 warnings** all pre-existing, none new	**15 / 15** test-node.mjs	**168 / 168**	**70 / 70** of which new 5 / 5	**33/33** compare.py

STEP (OCCT)	native ↔ Wasm
47/47 verify.py	**9 / 9** identical scenes

1 Scope

- **F1** `shell` of a filleted body (dose_shell.scad was red), **F2** `surface(points)` with 2 × 2 or 3 × 3 points, **F3** seam and pole edges as separate edges – the three bugs found in report 4, section 7. In addition **F4**: fase_chamfer.scad was red as well (`union(chamfer = 0.5)` selected eight instead of two edges).
- **Variable radius** (spec "Variable radius (stretch goal)"): `fillet(r = function (p) …) { … }` and `difference/union/intersection(fillet = function (p) …)`; radius as a function of position, evaluated at stations along each edge, hint "r 2–4", error cases as for the constant radius. New example `examples/spec/variabler_radius.scad`.
- **SMLib**: fix 1ed8669 (pointer maps in insertion order) from the fork branch `capi-solidcraft` as patch `smlib/patches/0001-pointer-maps-insertion-order.patch` on top of 7eba8aa (`smlib/fetch.sh` applies it). **Facade ABI 4** (`sc_version()` = 4): edge flags in `sc_edges`, `sc_law_stations`, `sc_fillet_var`; kernel ID `smlib-7eba8aa+1/facade-4` (new cache keys).

2 Test environment

Operating system	Linux 6.18.44-fc-v153 (x64)
CPU	Intel(R) Xeon(R) Processor @ 2.10GHz · 4 cores
Rust	rustc 1.97.0 (2d8144b78 2026-07-07)
Node.js	v22.22.0
Emscripten	emcc (Emscripten gcc/clang-like replacement + linker emulating GNU ld) 6.0.13 (e13ee988ce3d87a738dd8f3661d02223308c412a)
Browser	Chromium (Playwright 1.56.1), WebGL via SwiftShader, 1440 × 900 px
SMLib	tangency-labs/smlib capi-tess @ 7eba8aa + smlib/patches/0001 (1ed8669 from capi-solidcraft, source only), facade ABI 4
Source	Branch worktree-agent-a1ce9f9f0bb82a038, commit 19f2e47

3 Bugs: cause, fix, evidence

Diagnostic and hint texts in the tables below are quoted verbatim from the program output (German by design; "Z." = line, "Fehler" = error, "Warnung" = warning, "Kanten" = edges, "gescheitert" = failed).

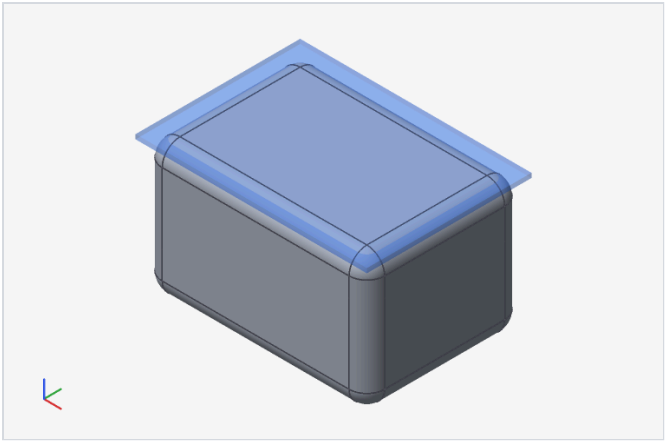
F1 shell of a filleted body

Cause Not the "no cavity" pre-check (it only concerns planar, opposite walls; the message "leaves no cavity" came after SMLib had returned an open body). SMLib's shell construction with an opened face (`SmApiShellBrepFull` → `SmOffsetExecutive::ShellBrep`, branch with opening via `SmMerge`) does not hold up on filleted bodies: depending on the options, `SM_ERR_NULL_POINTER` in `SmMerge.cpp:672`, an open body made of the 26 input faces, or a closed body whose border faces are missing and whose inner roundings are replaced by extended faces (11–15 faces fail the validity check, volume off by 0.08 %). Which case occurred depended on the address order of SMLib's pointer maps (hence different natively and in Wasm). The *closed* inward offset of the same body (`SmApiOffsetBrepFull`, −t), in contrast, is exact (volume = Minkowski formula to 10^{-7}).

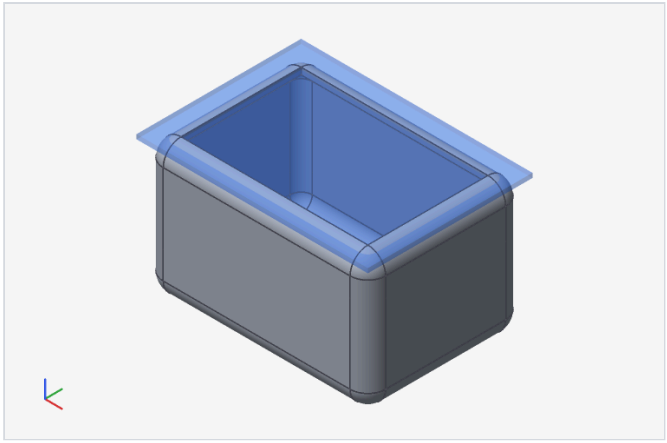
Fix `sc_shell` tries SMLib's shell; if it fails or self-overlaps, the facade builds the shell from booleans: $body - O - P_1 - \ldots - P_k$, O being the closed inward offset, P_i a prism over the offset face of each opened (planar) face, from just inside the cavity to beyond the body (thickened single face). The side walls of the prism go through the border of the offset face: the opening border is the band between the opened face and its offset, as with SMLib's own shell. Booleans go through the facade's checked booleans, the result through the validity check. In addition patch 0001 (1ed8669): SMLib's pointer maps return keys in insertion order – results no longer depend on the heap layout.

Evidence Volume against the closed form (rounded cuboid minus inner rounded cuboid with r − t minus the wall band under the planar top face), STEP check, identical scenes native and Wasm (section 5), Rust tests `shell_of_rounded_boxes`, `dose_shell_example_is_a_hollow_box`, `shell_of_a_rounded_box_open_at_the_top`, Node test, Playwright E03.

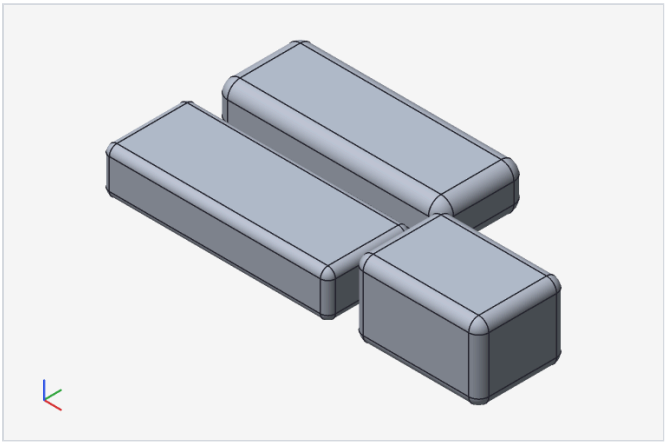
Case	before (cad check)	expected	after	rel. dev.	STEP	Diagnostics after
90 × 34 × 18, r 3, wall 1.6	Z.2: Fehler (Kernel): shell gescheitert (1.6 mm): shell produced an invalid (self-overlapping) solid at 15 face(s); parameter too large?	10 908.088 mm³	10 908.088 mm³	2.0e-10	54 faces	ok
90 × 34 × 18, r 5, wall 1.6	Z.2: Fehler (Kernel): shell gescheitert (1.6 mm): shell: wall thickness 1.600000 leaves no cavity (SMLib returned no closed solid)	10 822.339 mm³	10 822.339 mm³	1.1e-9	54 faces	ok
50 × 35 × 30, r 4, wall 1.5	Z.2: Fehler (Kernel): shell gescheitert (1.5 mm): shell produced an invalid (self-overlapping) solid at 12 face(s); parameter too large?	9 245.011 mm³	9 245.011 mm³	1.8e-9	54 faces	ok
90 × 34 × 18, r 6, wall 1.6	Z.2: Fehler (Kernel): shell gescheitert (1.6 mm): shell: wall thickness 1.600000 leaves no cavity (SMLib returned no closed solid)	10 754.828 mm³	10 754.828 mm³	7.3e-10	54 faces	ok
dose_shell.scad (with lid)	Z.9: Fehler (Kernel): shell gescheitert (1.5 mm): shell produced an invalid (self-overlapping) solid at 11 face(s); parameter too large?	–	12 697.900 mm³	–	80 faces	ok



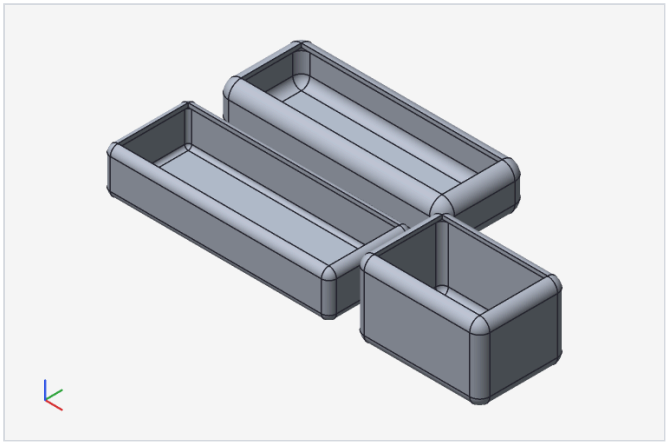
before: dose_shell.scad, module dose (cursor on shell, line 9)



after: dose_shell.scad, module dose (cursor on shell, line 9)



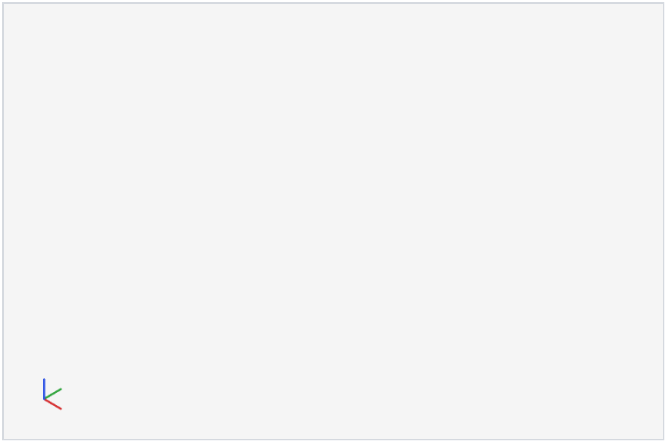
before: the three cases from report 4



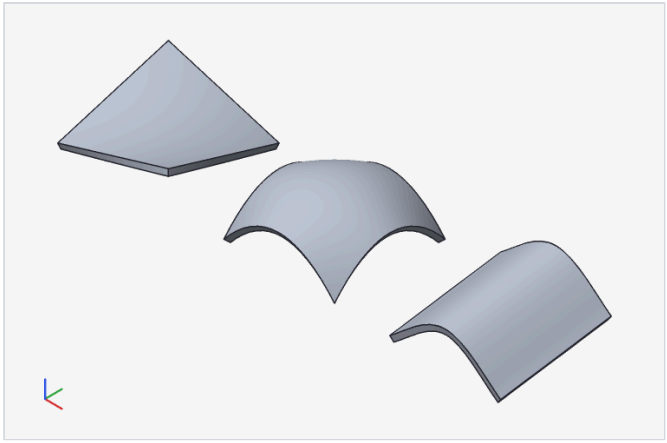
after: the three cases from report 4

F2 surface(points) with 2 × 2 or 3 × 3 points

Cause	`SmApiCreateSurfaceFromOrderedPoints` calls `SmBSplineSurface::ApproximatePoints` with a fixed degree 3 in both directions; that requires at least 4 points per direction, otherwise `SM_ERR`.
Fix	`sc_nurbs_surface` calls `ApproximatePoints` itself, with degree min(3, n − 1) separately per direction: bilinear for 2, quadratic for 3, cubic from 4 (unchanged from 4 × 4). With as many control points as points, the least-squares fit interpolates the points.
Evidence	`surfaces_get_the_degree_their_point_count_allows` (degrees 1/1, 2/2, 1/3, 3/1, 3/3 in the B-rep, points on the surface), `surfaces_from_two_and_three_points_per_direction` (planar grids 2 × 2 … 4 × 4: area 400 mm²), Node test, Playwright E05.
Messages	before: Z.1: Fehler (Kernel): surface gescheitert: SmApiCreateSurfaceFromOrderedPoints failed: SM_ERR (1001) Z.2: Fehler (Kernel): surface gescheitert: SmApiCreateSurfaceFromOrderedPoints failed: SM_ERR (1001) Z.3: Fehler (Kernel): surface gescheitert: SmApiCreateSurfaceFromOrderedPoints failed: SM_ERR (1001) after: –



before: 2 × 2, 3 × 3 and 2 × 5 points, thickened



after: 2 × 2, 3 × 3 and 2 × 5 points, thickened

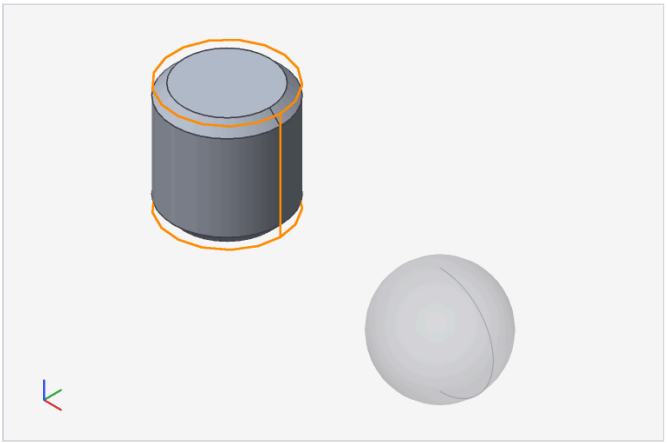
F3 Seam and pole edges counted as edges

Cause For `$fillet/$chamfer` the engine selected all edges of the primitive, and in zones all edges in the zone. SMLib's cylinder has a seam (same face on both sides), the sphere a seam from pole to pole. The facade skipped such edges when filleting (`GetBlendEdgeuse() == NULL`), but they were still counted and drawn; for the sphere no edge remained → error.

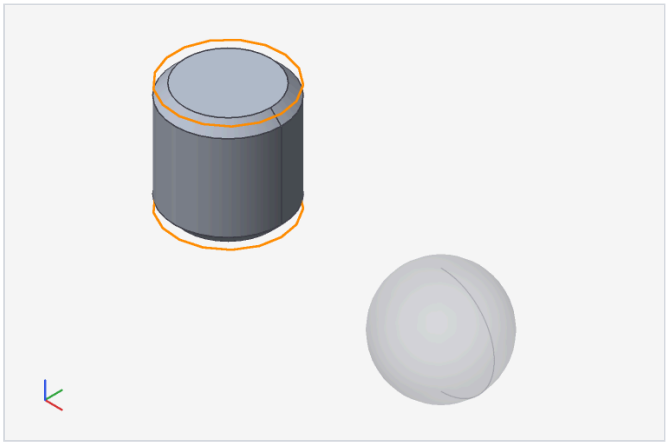
Fix `sc_edges` reports flags per edge, `SC_EDGE_SMOOTH` = nothing to fillet (seam, pole, tangentially continuous transition within 2°, open border; SMLib's own criterion). `EdgeInfo.smooth`; the engine never selects such edges – for `$fillet/$chamfer`, in zones and for `fillet =` on booleans. A primitive without an edge to fillet yields "0 edges" as a warning without effect. The tangent borders of earlier roundings no longer count either (nested blocks).

Hints Cylinder before: cylinder: 3 Kanten · after: cylinder: 2 Kanten
Sphere before: sphere: 1 Kante · after: sphere: 0 Kanten
Messages before: Z.2: Fehler (Kernel): Verrundung gescheitert (1 mm): fillet: none of the edges can be blended (smooth, seam or open edges) · after: Z.2: Warnung (Kernel): 0 Kanten: das Primitiv hat keine Kanten zum Verrunden

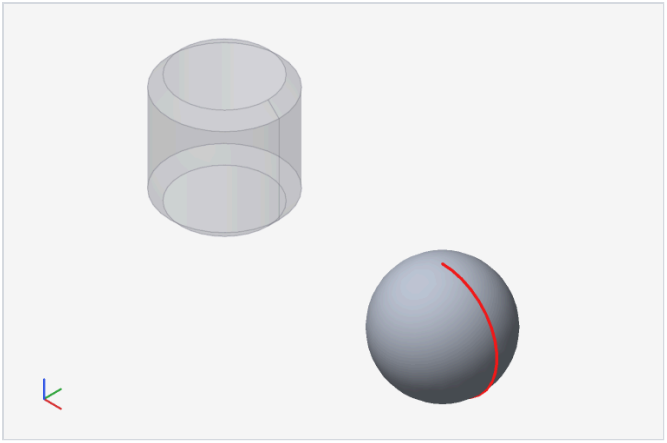
Evidence `seams_and_poles_are_not_edges` (cylinder 2 edges, volume with two chamfer rings; sphere 0 edges, warning, volume unchanged; zone around the cylinder 2 edges; union of two equal cylinders: 0 new edges), `tangent_edges_of_rounds_are_not_selected`, `edges_flag_seams_poles_and_tangent_edges_as_smooth`, Playwright E04 and C05.



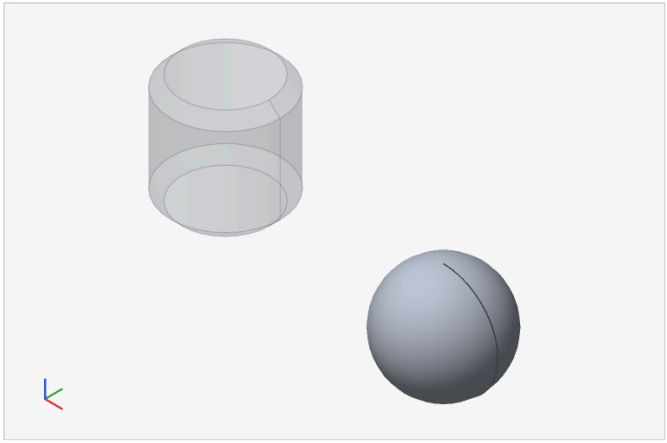
before: cylinder(..., $chamfer = 1), cursor line 1 (selected edges orange)



after: cylinder(..., $chamfer = 1), cursor line 1 (selected edges orange)



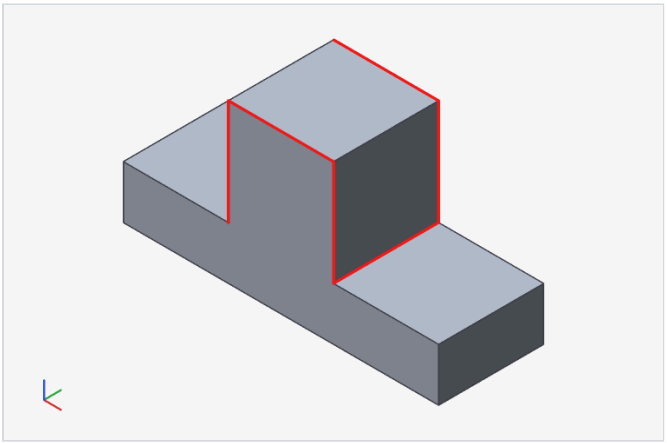
before: sphere(5, $fillet = 1), cursor line 2



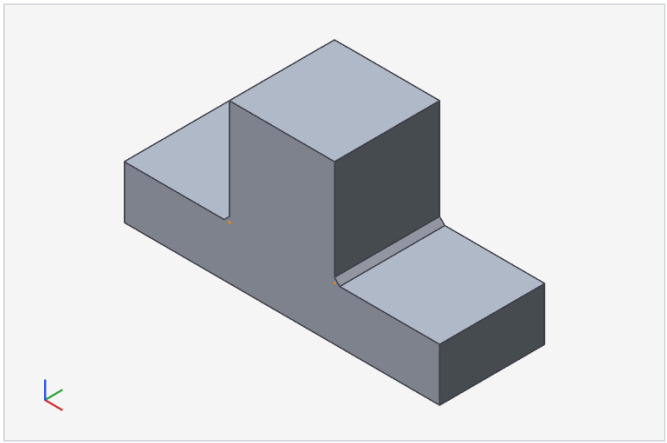
after: sphere(5, $fillet = 1), cursor line 2

F4 fase_chamfer.scad: union(chamfer) selected eight edges

Cause	"New edges" of a boolean operation were edges between faces of different operands (face tags). If two operands share a plane (plate and tower with the same front and back face), SMLib merges the faces into one with the tag of one operand; the six old tower edges on the front and back then separated faces of different operands and were considered new. The chamfer on these edges failed (`SM_ERR`).
Fix	An edge that lies on an edge of an operand is not new (sample points on the edge chain of an operand, tolerance 10^{-6} of the model size).
Hints	before: union: 8 Kanten · after: union: 2 Kanten Messages before: Z.1: Fehler (Kernel): Fase gescheitert (0.5 mm): chamfer failed: SM_ERR (1001) · after: –
Evidence	`coplanar_merged_faces_keep_old_edges_old` (2 edges, volume 2502.5 mm³), fase_chamfer.scad without errors (stage 1, STEP, run time table).



before: union(chamfer = 0.5) from fase_chamfer.scad, cursor line 1



after: union(chamfer = 0.5) from fase_chamfer.scad, cursor line 1

4 Variable radius

Architecture: evaluable description instead of a callback

The edges at which the radius is needed only exist once the kernel has built the body – long after the evaluation, in the engine. Two options: a callback from the engine into the evaluation, or the evaluation supplies an evaluable description of the function. The description was chosen: `cad-eval` translates the function literal into a **radius law** (`cad_ir::Law`), a small pure expression over `p`. Whatever does not depend on `p` (captured variables, grid planes, list

comprehensions, arbitrary builtins) is evaluated at that point and becomes a constant; calls to user functions and function values are inlined (parameters as local slots, depth ≤ 16); what remains are literals, arithmetic, comparisons, `&& || ! ?:`, `let`, indices, `.x/.y/.z`, vectors and the math builtins (`sin … lookup`).

- The law is part of the tree (serializable, part of the cache key): a changed captured variable changes the key, the same source hits the cache. A callback would need its own key for the closure including its environment.
- The engine still does not depend on `cad-eval` and its AST lifetime; Wasm and native evaluate the same data object, also from the cache, without re-evaluating the source.
- Limitation: only the pure expression core can depend on `p` (no strings, no list comprehensions over `p`, no recursion over `p`); such cases are errors at the function with their own message.

Procedure

1. Evaluation: `fillet(r = function …)` → `Op::Blend { law }`, `difference(fillet = function …)` → `Op::Boolean { fillet_law }`. `chamfer` and `$fillet` with a function: warning, not supported.
2. Engine: edge selection as for the constant radius (zones, new edges, without smooth edges); per edge *m* stations (about one per 1.5 mm, 4 to 32) from `Kernel::law_stations`; the law is evaluated at each station in the coordinate system of the fillet/boolean statement. Radius ≤ 0 or not a number → error with value and position, edge red, body unchanged. All values equal → constant fillet (same result as `fillet(r)`). Otherwise `Kernel::fillet_variable`; kernel errors as for the constant radius, with the range ("Verrundung gescheitert (8–18 mm): …"). Hint at the end of the line: "r *min–max*" of the station values.
3. Facade (`sc_law_stations`, `sc_fillet_var`): the law of an edge is a clamped B-spline of degree min(3, m − 1) over the normalized edge parameter with uniform interior knots, control radius *j* at the Greville point t_j (that is where the stations lie). It stays between the smallest and largest value, is exact for constant and linear profiles and takes the first and last value at the edge ends. As `SmBSplineFilletLaw`, the law curve carries the radius as x and t itself as y (control points (r_j, t_j), i.e. y(t) = t), so that SMLib can extend it beyond the edge ends at corners. Per edge SMLib's variable-radius solver (`SM_FS_VARIABLE_RADIUS`), edges with equal values use the constant one.

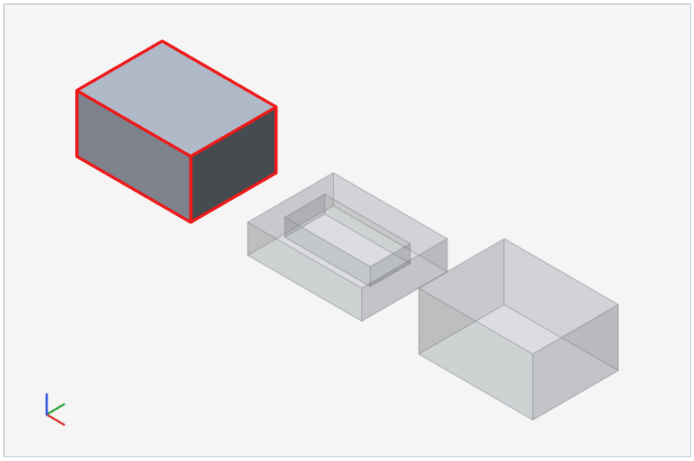
Measurements

x (mm)	Law 1 + x/15	measured (start of the top face)	Dev.
2.0	1.1333	1.1227	-0.94 %
7.5	1.5000	1.4878	-0.81 %
15.0	2.0000	1.9859	-0.71 %
22.5	2.5000	2.4843	-0.63 %
28.0	2.8667	2.8498	-0.59 %

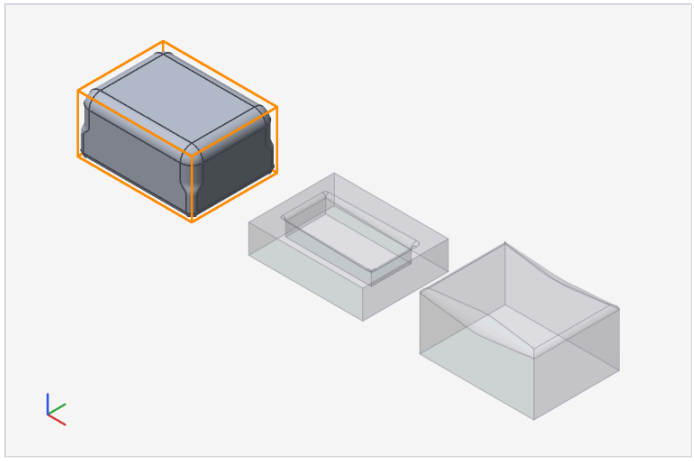
One edge of a 30 × 20 × 20 cuboid with a linear law: radius measured via point classification just below the top face (that is where the rounding starts; with a growing radius the tangent line lies slightly behind r(x)). Volume against the closed form 12000 − (1 − π/4)·130: deviation < 10^{-5} (Rust and Node test). Step law `p.z < 10 ? 2 : 4` on all 12 edges of a 30 × 30 × 20 cuboid: 17 346.308 mm³, between r = 4 (16 986.950 mm³) and r = 2 (17 736.024 mm³). Pocket border with `1 + p.x / 40`: 9 434.560 mm³ between r = 1.875 (9 411.622 mm³) and r = 1.125 (9 455.670 mm³). variabler_radius.scad: 56 527.548 mm³, STEP 55 faces, `cad export` 1660 ms.

Line	before	after
11	fillet: 12 Kanten	fillet: 12 Kanten · r 2–4
16	difference: –	difference: 4 Kanten · r 1.125–1.875
22	fillet: 1 Zone · 4 Kanten	fillet: 1 Zone · 4 Kanten · r 1.5–3.9

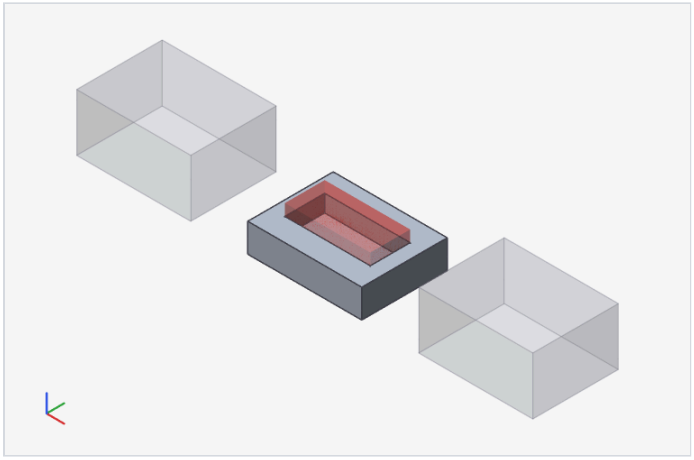
Diagnostics	Z.11: Warnung (Auswertung): variabler Radius `r` (Funktion) noch nicht unterstützt Z.16: Warnung (Auswertung): variabler Radius `fillet` (Funktion) noch nicht unterstützt Z.22: Warnung (Auswertung): variabler Radius `r` (Funktion) noch nicht unterstützt Z.11: Fehler (Kernel): Verrundung gescheitert (0 mm): invalid input: fillet size must be > 0 Z.22: Fehler (Kernel): Verrundung gescheitert (0 mm): invalid input: fillet size must be > 0	–



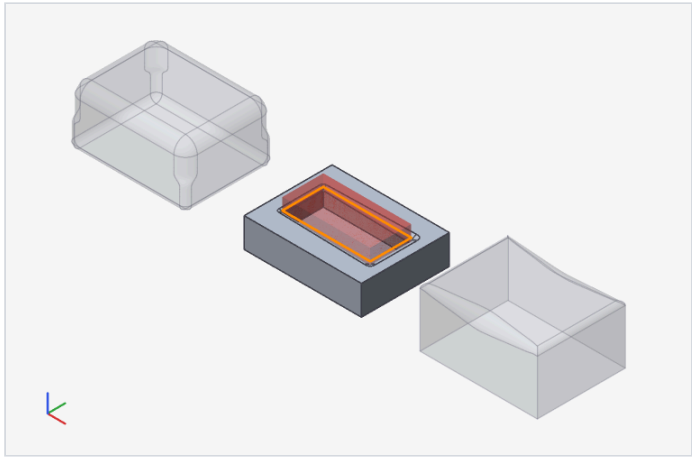
before: variabler_radius.scad, cursor line 11 (step law)



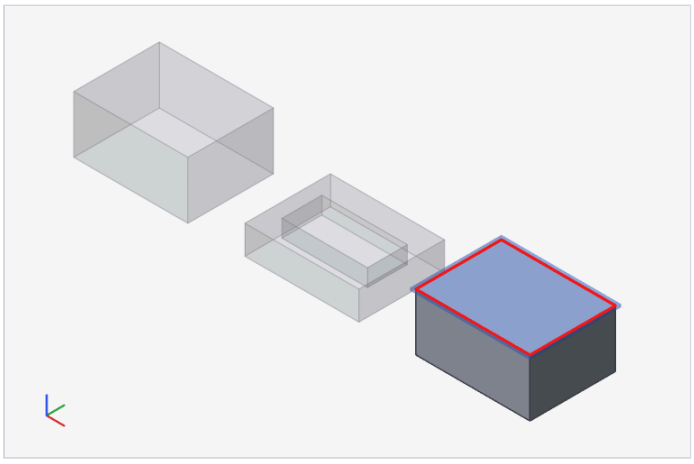
after: variabler_radius.scad, cursor line 11 (step law)



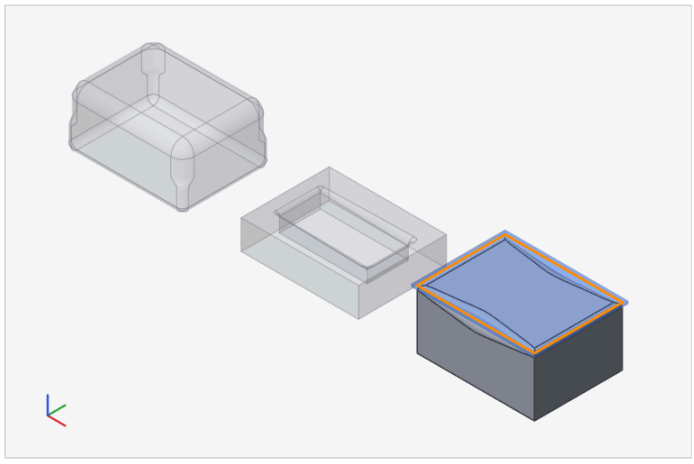
before: line 16: difference(fillet = function (p) 1 + p.x / 40)



after: line 16: difference(fillet = function (p) 1 + p.x / 40)



before: line 22: lookup law with zone at the lid edges



after: line 22: lookup law with zone at the lid edges

5 Native and Wasm

Scene from `cad eval --json` (native) against `web/scripts/dump-scene.mjs` (browser engine in Node: cad-wasm + smlib.wasm) per node: status, hints, number of selected edges, bounding box (10^{-4} mm), messages, and the diagnostics.

File	Nodes	of which errors	Result	Deviation
F1 (three cases)	12	0	identical	
F1 (r = 6)	4	0	identical	
dose_shell.scad	9	0	identical	
F2	9	0	identical	
F3	4	0	identical	
F4	5	0	identical	
variabler_radius.scad	12	0	identical	
griffschale.scad	19	0	identical	
fillet_zonen.scad	16	0	identical	

6 Run times of cad export (all examples)

`cad export <file> -o x.stl` (release, wall-clock time incl. startup, one run per file, sequentially) before (state before this task) and after. Total over 47 common files: before 21743 ms, after 23587 ms. Exit 1 = diagnostics with errors (before: dose_shell, fase_chamfer).

File	before	after	Δ
spec/dose_shell.scad	988 ms (Exit 1)	1925 ms	+937 ms
spec/fase_chamfer.scad	319 ms (Exit 1)	394 ms	+75 ms
spec/fillet_zonen.scad	429 ms	439 ms	+10 ms
spec/flasche.scad	715 ms	673 ms	-42 ms
spec/freiformdeckel.scad	3657 ms	3868 ms	+211 ms
spec/gehaeuse.scad	40 ms	56 ms	+16 ms
spec/griffschale.scad	584 ms	693 ms	+109 ms
spec/kabelfuehrung.scad	426 ms	595 ms	+169 ms
spec/knauf_mirror.scad	2308 ms	2224 ms	-84 ms
spec/lochplatte.scad	957 ms	1127 ms	+170 ms
spec/loft_sweep.scad	811 ms	698 ms	-113 ms
spec/lueftungsgitter.scad	549 ms	519 ms	-30 ms
spec/winkel_anker.scad	242 ms	326 ms	+84 ms
openscad-compat/01_cube_variants.scad	30 ms	38 ms	+8 ms
openscad-compat/02_sphere_variants.scad	774 ms	737 ms	-37 ms
openscad-compat/03_cylinder_variants.scad	150 ms	176 ms	+26 ms
openscad-compat/04_cone_variants.scad	103 ms	117 ms	+14 ms
openscad-compat/05_nested_booleans.scad	1301 ms	1467 ms	+166 ms
openscad-compat/06_hole_grid.scad	1304 ms	1128 ms	-176 ms
openscad-compat/07_crown_ring.scad	783 ms	701 ms	-82 ms

openscad-compat/08_module_children.scad	262 ms	312 ms	+50 ms
openscad-compat/09_rotate_mirror.scad	81 ms	57 ms	-24 ms
openscad-compat/10_scale_multmatrix.scad	29 ms	26 ms	-3 ms
openscad-compat/11_linear_extrude_basic.scad	14 ms	15 ms	+1 ms
openscad-compat/12_linear_extrude_twist.scad	390 ms	480 ms	+90 ms
openscad-compat/13_rotate_extrude_ring.scad	521 ms	585 ms	+64 ms
openscad-compat/14_pipe_bend.scad	486 ms	484 ms	-2 ms
openscad-compat/15_vase.scad	164 ms	168 ms	+4 ms
openscad-compat/16_polygon_holes.scad	40 ms	45 ms	+5 ms
openscad-compat/17_polyhedron.scad	11 ms	17 ms	+6 ms
openscad-compat/18_functions_let.scad	43 ms	78 ms	+35 ms
openscad-compat/19_hex_nut.scad	96 ms	162 ms	+66 ms
openscad-compat/20_box_with_lid.scad	546 ms	710 ms	+164 ms
openscad-compat/21_angle_bracket.scad	267 ms	303 ms	+36 ms
openscad-compat/22_gear.scad	340 ms	320 ms	-20 ms
openscad-compat/23_shelf_holder.scad	176 ms	212 ms	+36 ms
openscad-compat/24_modifiers.scad	45 ms	49 ms	+4 ms
openscad-compat/25_root_modifier.scad	42 ms	44 ms	+2 ms
openscad-compat/26_if_else_variants.scad	273 ms	268 ms	-5 ms
openscad-compat/27_spacers_2d.scad	414 ms	367 ms	-47 ms
openscad-compat/28_cable_clip.scad	177 ms	178 ms	+1 ms
openscad-compat/29_assert_strings.scad	64 ms	61 ms	-3 ms
openscad-compat/30_each_ranges.scad	80 ms	67 ms	-13 ms
openscad-compat/31_phone_stand.scad	34 ms	33 ms	-1 ms
openscad-compat/32_low_fn_shapes.scad	42 ms	39 ms	-3 ms
openscad-compat/33_global_fn.scad	256 ms	253 ms	-3 ms
projekt-gehaeuse/gehaeuse.scad	380 ms	353 ms	-27 ms
spec/variabler_radius.scad	–	1660 ms	–

7 Tests

- **cargo test --workspace** (3 runs, no failures between runs): 298 passed, 0 failed · 298 passed, 0 failed · 298 passed, 0 failed.
- **cargo clippy --workspace --all-targets**: 3 warnings, 0 errors; the warnings are in unchanged files (`crates/cad-eval/src/edit.rs`, `crates/cad-eval/tests/project_openscad.rs`, `crates/cad-engine/src/parts.rs`) and existed before – none new.
- **node crates/kernel-smlib/js/test-node.mjs** (smlib.wasm via the bridge): 15 / 15 – new: edge flags, surface 2 × 2/3 × 3/2 × 5, shell of a filleted cuboid (volume to 10^{-6}), variable radius (stations, linear law, radius ≤ 0).
- **OpenSCAD comparison** (compare.py): `33/33 files match`.
- **STEP** with OpenCASCADE (`tests/step/verify.py` over all examples): `47/47 passed`.
- **Vitest**: 168 / 168; **pnpm typecheck** without errors.
- **Playwright**: 70 / 70 (existing scenarios 65 / 65, new 5 / 5).

ID	Scenario (web/e2e/ui-e.spec.ts)	Result	Duration	Checked
E01	Variable radius: hint "r 2–4", edges orange, result filleted	passed	5.7 s	• status ok, hints ["12 Kanten","r 2–4"], ["4 Kanten","r 1.125–1.875"], ["1 Zone","4 Kanten","r 1.5–3.9"] • selected edges drawn 12 / 4 / 4, pixels of the orange edges (step): 4304 • triangles of the filleted box: 2228
E02	Variable radius: radius ≤ 0 and too large a radius behave as for the constant radius	passed	2.4 s	• status error/error, triangles 12/12 • message: Verrundung gescheitert (variabler Radius): Radius -5 ≤ 0 bei [0, 20, 20]
E03	shell of a filleted body: dose_shell.scad and the three cases from report 4	passed	8.8 s	• dose: 2332 triangles, 1 face opened (view and engine) • cases: status ok/ok/ok, 1/1/1 face opened each; middle 2332 triangles
E04	Seam and pole edges do not count: cylinder 2 edges, sphere 0 edges	passed	2.9 s	• edges drawn 2 / 0 / 2 • sphere: status warning, line yellow
E05	surface from 2 × 2 and 3 × 3 points	passed	2.8 s	• both surface nodes ok

Rust test program (last run)	passed	new tests
tests/cli.rs	9 / 9	–
tests/projects.rs	3 / 3	–
src/lib.rs	7 / 7	–
tests/draft.rs	3 / 3	–
tests/engine.rs	25 / 25	–
tests/kernel_fixes.rs new	12 / 12	`coplanar_merged_faces_keep_old_edges_old`, `dose_shell_example_is_a_hollow_box`, `law_changes_change_the_cache_key`, `seams_and_poles_are_not_edges`, `shell_too_thick_still_fails`, `tangent_edges_of_rounds_are_not_selected`, `shell_of_rounded_boxes`, `variable_radius_captured_values_user_functions_and_booleans`, `surfaces_from_two_and_three_points_per_direction`, `variable_radius_errors_like_constant_radius`, `variable_radius_linear_law_on_one_edge`, `variable_radius_step_law_between_constant_radii`
tests/spec_examples.rs	4 / 4	–
tests/expr.rs	25 / 25	–
tests/grids.rs	14 / 14	–
tests/handles.rs	26 / 26	–
tests/law.rs new	5 / 5	`step_law_and_text`, `boolean_parameter_and_trig`, `law_is_part_of_the_tree_data`, `captured_values_functions_let_and_builtins`, `unsupported_parts_are_errors`
tests/limits.rs	8 / 8	–
tests/openscad_compat.rs	1 / 1	–
tests/openscad_deviations.rs	10 / 10	–
tests/project.rs	16 / 16	–
tests/project_openscad.rs	2 / 2	–
tests/tree.rs	24 / 24	–
src/lib.rs	2 / 2	`law::tests::step_law`, `law::tests::let_lookup_and_errors`
src/lib.rs	5 / 5	–

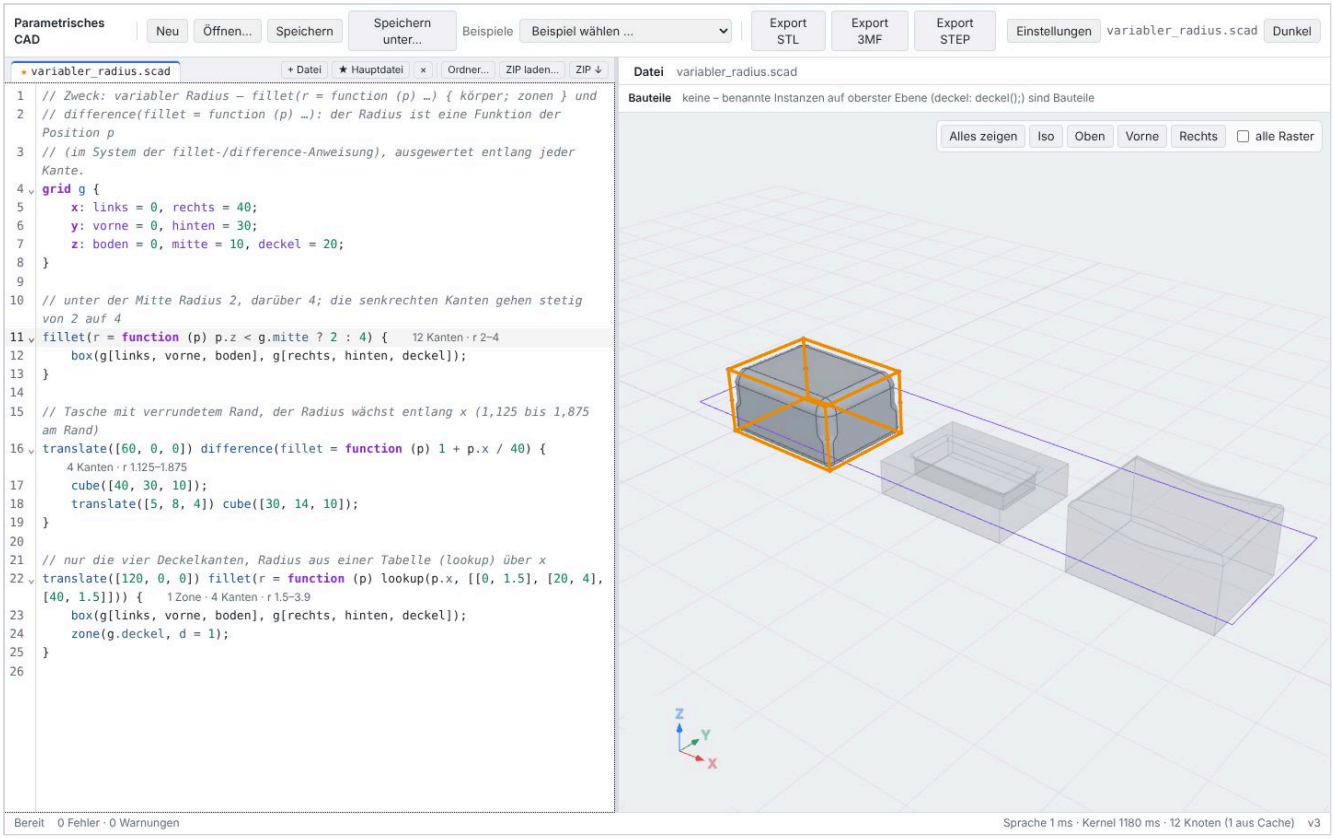
tests/render.rs	4 / 4	–
tests/project.rs	5 / 5	–
tests/protocol_ts.rs	1 / 1	–
tests/session.rs	8 / 8	–
tests/watchdog.rs	2 / 2	–
src/lib.rs	3 / 3	–
tests/writer.rs	6 / 6	–
src/lib.rs	19 / 19	–
tests/openscad_compat.rs	1 / 1	–
tests/spec_models.rs	1 / 1	`spec_models_stage1`
tests/tree_bounds.rs	1 / 1	–
tests/brep.rs	5 / 5	–
tests/determinism.rs	1 / 1	–
tests/facade4.rs new	5 / 5	`edges_flag_seams_poles_and_tangent_edges_as_smooth`, `law_stations_and_a_linear_law`, `facade_version`, `surfaces_get_the_degree_their_point_count_allows`, `shell_of_a_rounded_box_open_at_the_top`
tests/handles.rs	1 / 1	–
tests/kernel.rs	33 / 33	–
tests/coverage.rs	1 / 1	–

8 Limitations

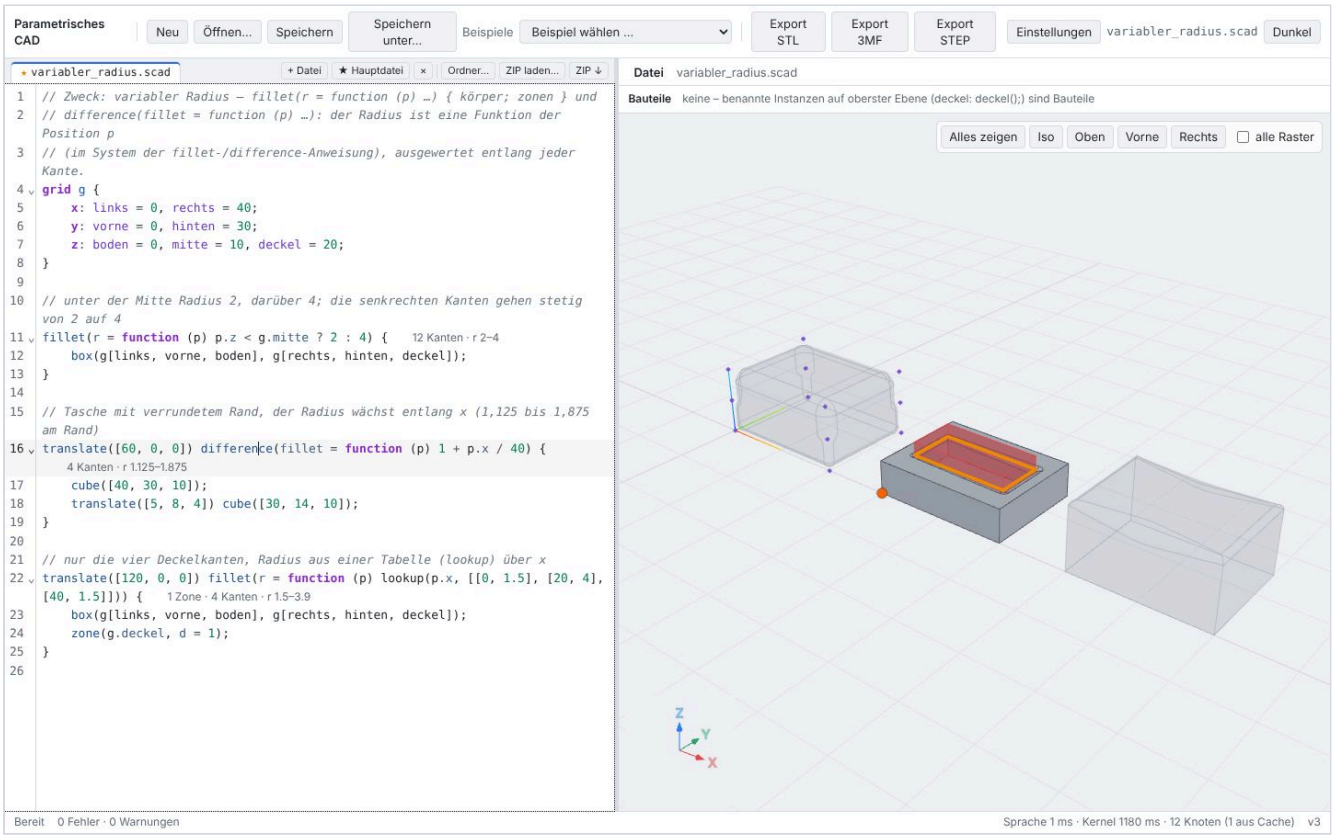
- **shell from booleans** only for planar opened faces (the prism is the thickened offset face); for curved opened faces SMLib's result or its error message remains. Closed shells (without a zone) still use SMLib's result if it is valid. The booleans cost time: dose_shell 988 ms (failing) → 1925 ms (section 6); in smlib.wasm (Node test, 90 × 34 × 18, r 3) 1758.4 ms.
- **Wall thickness ≥ rounding radius** (r ≤ t, e.g. 30 × 20 × 10 with r 1, wall 2): the inward offset would be sharp-edged; SMLib's offset fails there in 7eba8aa (`SM_ERR`, fork commit a252570 not adopted), and the shell remains an error as before (the message names both approaches).
- **Variable radius:** SMLib's variable-radius solver is sensitive. Steep profiles with a large radius at pocket corners fail ("self-overlapping", e.g. pocket border with 1 + p.x/20, 1.25–2.75), as do circular edges with a varying radius (`SM_ERR`); the example therefore uses 1 + p.x/40. Jumps (`p.z < 10 ? 2 : 4` on vertical edges) are smoothed over about three station spacings; the hint shows the range of the station values (lookup peak 4 → "3.9", because no station lies exactly at x = 20). Position `p` in the coordinate system of the fillet/boolean statement. No variable radius for `chamfer` and `$fillet` (warning).
- **Smooth edges:** SMLib's criterion (faces tangent within 2°) – nearly tangent edges are never selected. The check "old edge of an operand" (F4) compares with the edge chains of the operands, for curved edges only as accurately as their sampling (there the earlier selection remains).
- **SMLib patch** as a working-copy patch on top of 7eba8aa (the fork commit is not on `capi-tess`); prebuilt libraries (`SMLIB_PREBUILT`) must be rebuilt with the patch (header change, otherwise ODR violation). A symlink `third_party/smlib` is only checked by `fetch.sh`.

9 Screenshots of the browser UI

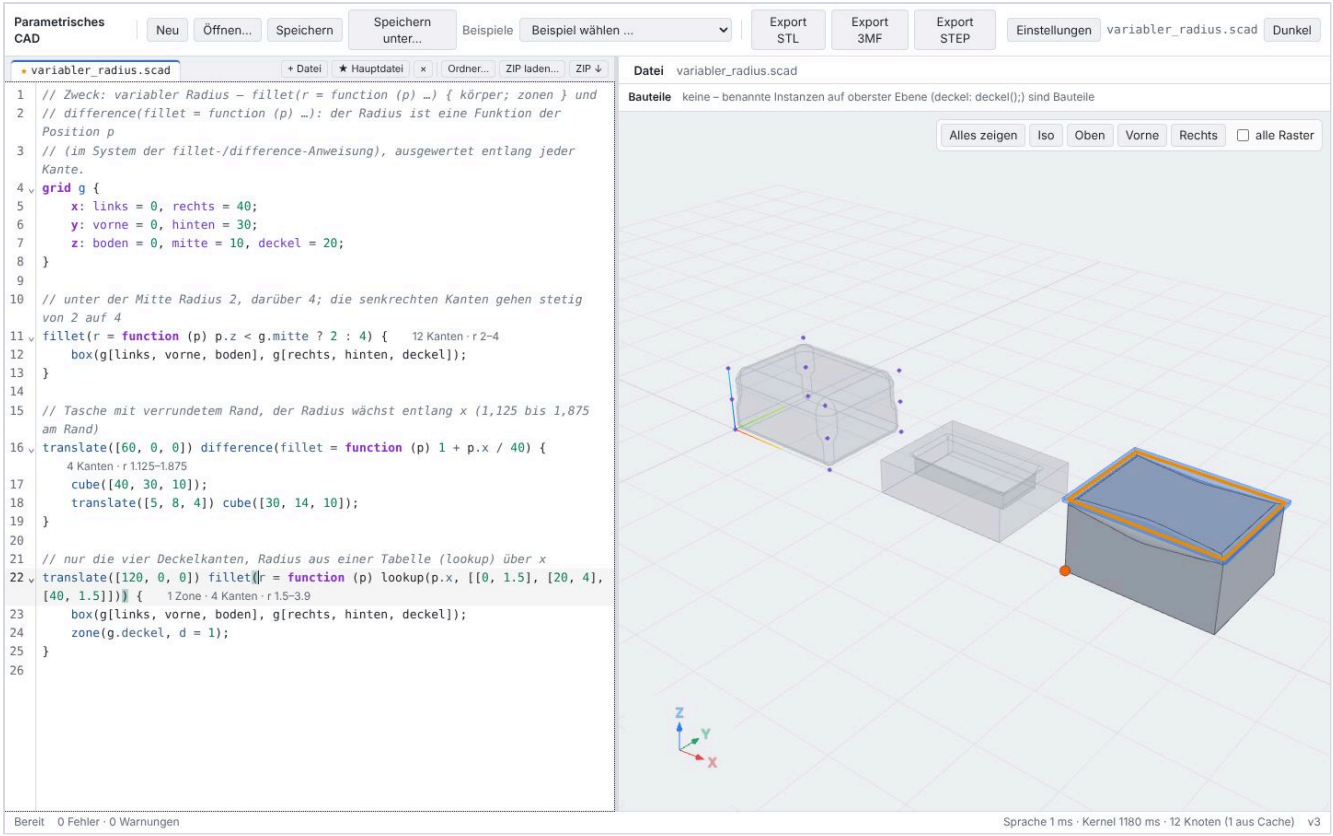
E01 Variable radius in the browser



Cursor on fillet(r = function (p) p.z < g.mitte ? 2 : 4)



Cursor on difference(fillet = function (p) 1 + p.x / 40)



Cursor on fillet(r = function (p) lookup(p.x, ...)) with zone

CODE

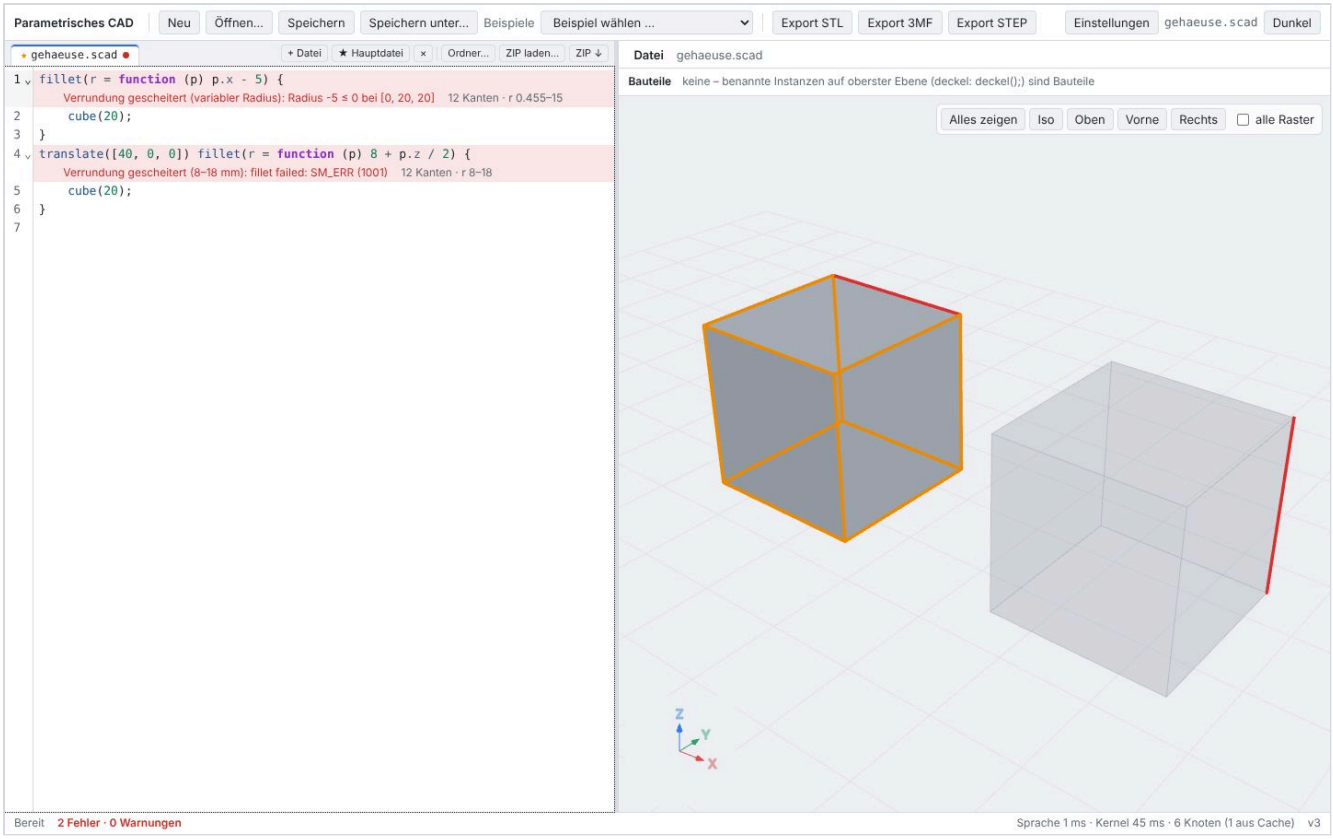
```
// radius 2 below the middle, 4 above; the vertical edges go continuously from 2 to 4
fillet(r = function (p) p.z < g.mitte ? 2 : 4) {
    box(g[links, vorne, boden], g[rechts, hinten, deckel]);
}

// pocket with a filleted border, the radius grows along x (1.125 to 1.875 at the border)
translate([60, 0, 0]) difference(fillet = function (p) 1 + p.x / 40) {
    cube([40, 30, 10]);
    translate([5, 8, 4]) cube([30, 14, 10]);
}

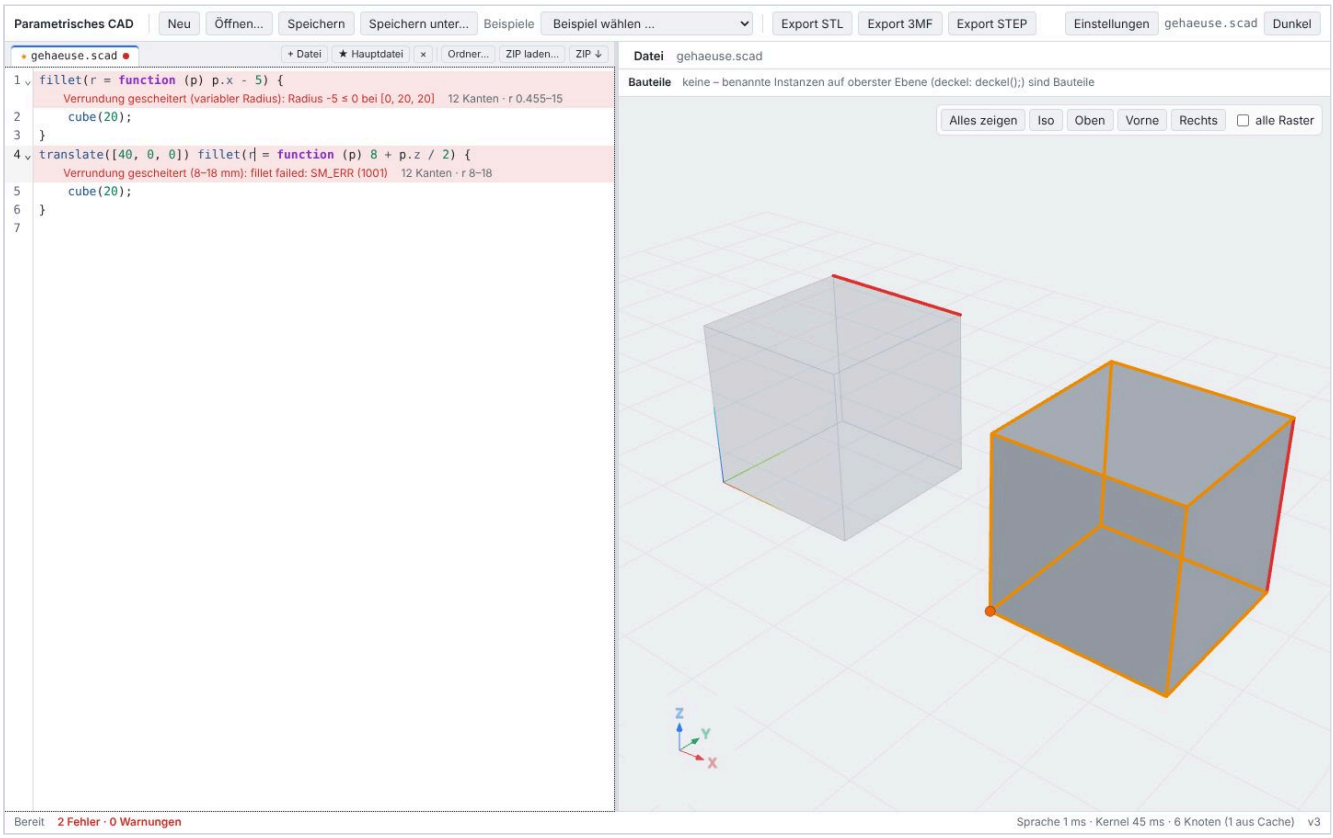
// only the four lid edges, radius from a table (lookup) over x
translate([120, 0, 0]) fillet(r = function (p) lookup(p.x, [[0, 1.5], [20, 4], [40, 1.5]])) {
    box(g[links, vorne, boden], g[rechts, hinten, deckel]);
    zone(g.deckel, d = 1);
}
```

variabler_radius.scad: the radius is a function of position. The end of the line shows the edge count and the radius range of the evaluated stations ("12 edges · r 2–4", "4 edges · r 1.125–1.875", "1 zone · 4 edges · r 1.5–3.9"); selected edges orange, zone blue, as for the constant radius.

E02 Error cases of the variable radius



Radius ≤ 0 on an edge



Radius 8–18 on a 20 mm cube

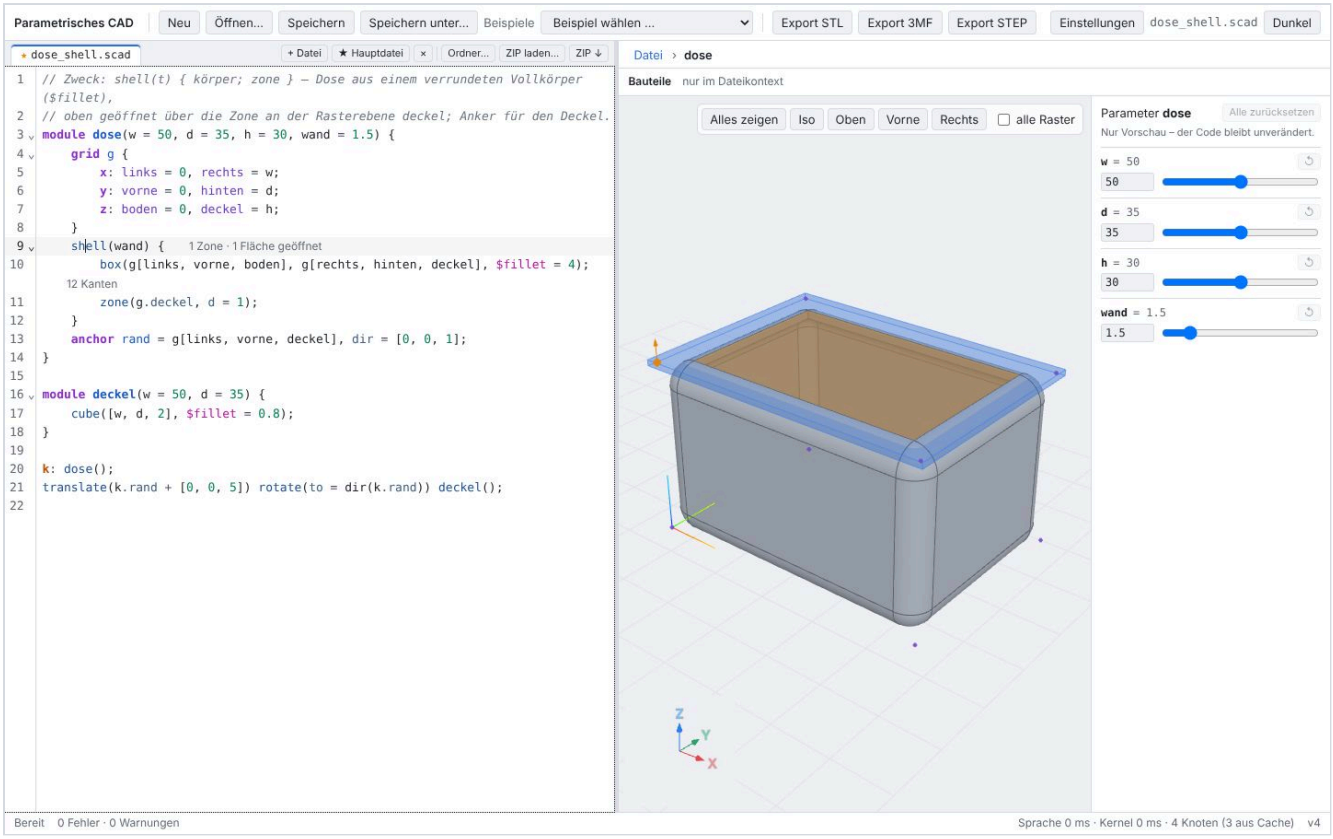
CODE

```
fillet(r = function (p) p.x - 5) {
    cube(20);
}
translate([40, 0, 0]) fillet(r = function (p) 8 + p.z / 2) {
```

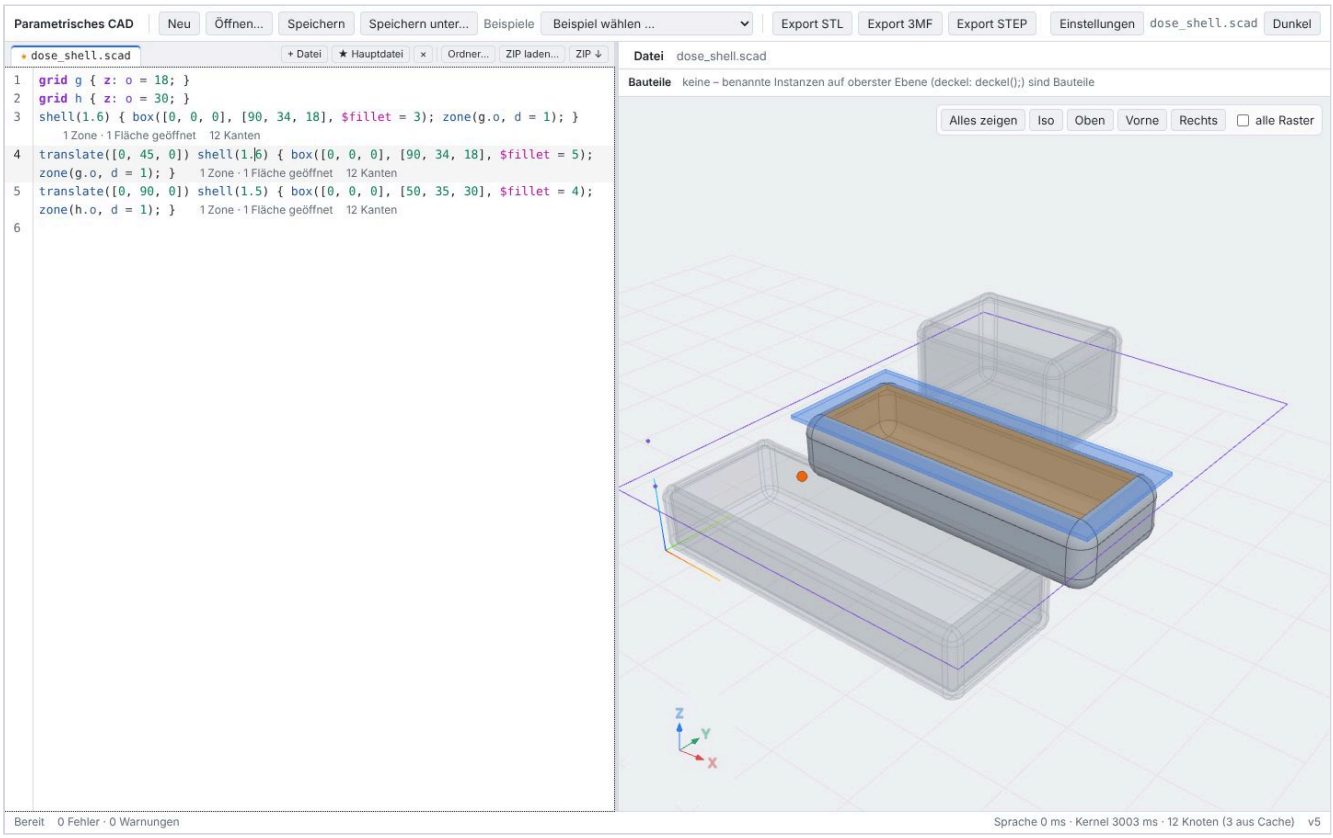
```
    cube(20);
}
```

Radius ≤ 0 at a station: line red, message with value and position, the edge red, the cube stays without fillet (12 triangles). Too large a radius: kernel message with the radius range, body unchanged – as for the constant radius.

E03 shell of filleted bodies



dose_shell.scad, cursor on shell(wand)



the three cases from report 4 (F1)

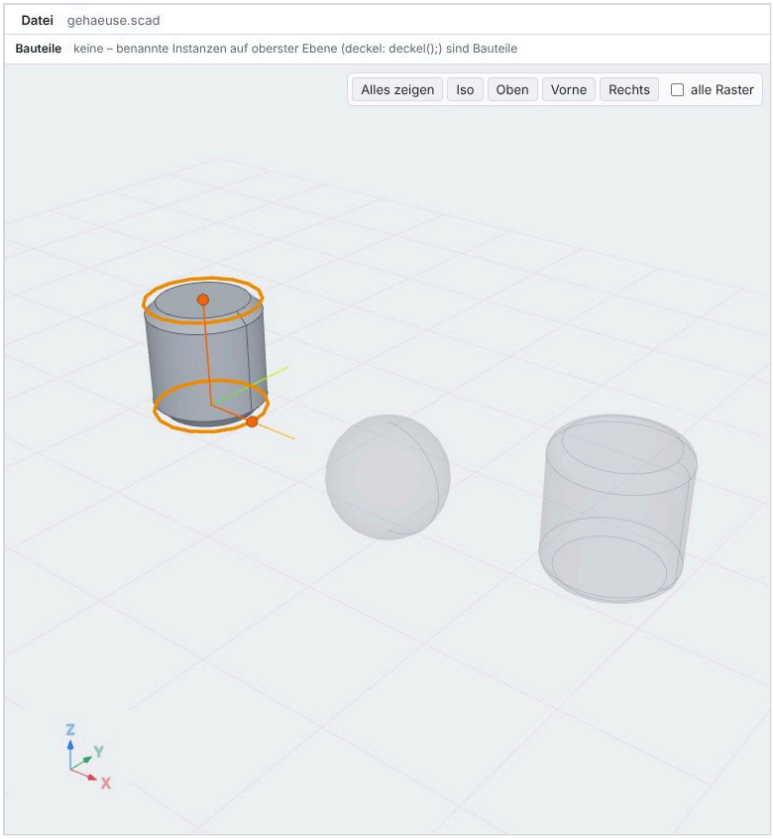
CODE

```
grid g { z: o = 18; }
grid h { z: o = 30; }
shell(1.6) { box([0, 0, 0], [90, 34, 18], $fillet = 3); zone(g.o, d = 1); }
```

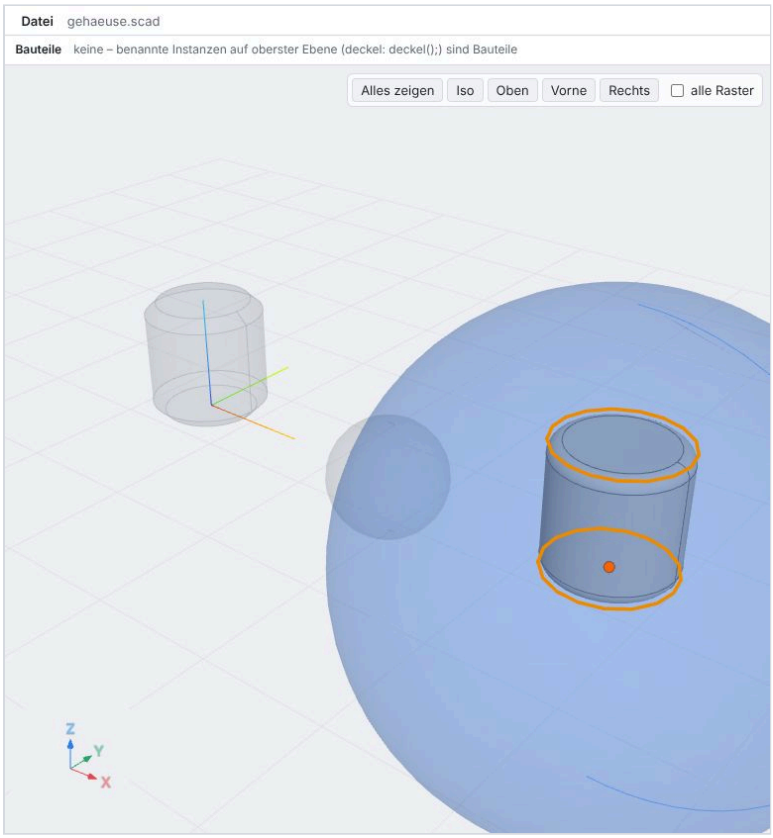
```
translate([0, 45, 0]) shell(1.6) { box([0, 0, 0], [90, 34, 18], $fillet = 5); zone(g.o, d = 1); }
translate([0, 90, 0]) shell(1.5) { box([0, 0, 0], [50, 35, 30], $fillet = 4); zone(h.o, d = 1); }
```

dose_shell.scad is green again: a closed hollow body with an opening at the top, "1 zone · 1 face opened". The three cases from report 4 (formerly SM_ERR_NULL_POINTER, "leaves no cavity", "self-overlapping") give hollow bodies.

E04 Seam and pole edges



cylinder(..., $chamfer = 1): top and bottom edge



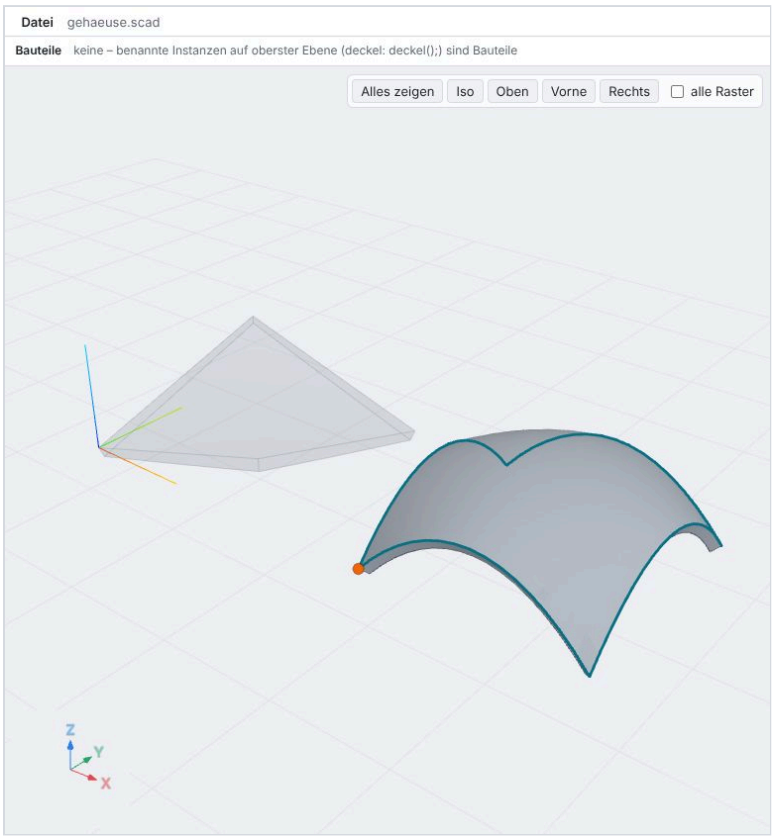
fillet(1) with a sphere zone around the cylinder

CODE

```
cylinder(h = 10, r = 5, $chamfer = 1);
translate([20, 0, 0]) sphere(5, $fillet = 1);
translate([40, 0, 0]) fillet(1) { cylinder(h = 10, r = 5); sphere(20); }
```

The seam of the cylinder and the seam/poles of the sphere are smooth and are never selected: $chamfer on the cylinder "2 edges" (before 3), $fillet on the sphere "0 edges" as a warning without effect (before: error), zone selection around the whole cylinder also 2 edges.

E05 surface from few points



bilinear (2 × 2) and quadratic (3 × 3)

CODE

```
thicken(1) surface(points = [[[0, 0, 0], [20, 0, 6]], [[0, 20, 6], [20, 20, 0]]]);
translate([30, 0, 0]) thicken(1) surface(points = [[[0, 0, 0], [10, 0, 6], [20, 0, 0]], [[0, 10, 6], [10,
10, 9], [20, 10, 6]], [[0, 20, 0], [10, 20, 6], [20, 20, 0]]]);
```

surface(points) with 2 × 2 (bilinear) and 3 × 3 points (quadratic) is created and can be thickened (before: SmApiCreateSurfaceFromOrderedPoints ... SM_ERR).